DESIGN DOCUMENT · PROPOSAL

Renting out Minecraft servers

What Cubeherd needs so that paying customers can run their own server on it: hardened containers, limits per server, safer accounts (two-factor, e-mail, registration) and clear rights.

Decisions taken: e-mail on sign-in and on registration, both switchable by an admin · limits per server (memory, CPU, disk), not per customer · disk limit like memory and CPU · customers may upload their own JARs · approving a registration assigns nothing · billing stays outside, an API follows later. Builds on `accounts-rbac-design.pdf` and `statistics-design.pdf`.

1. Summary

Cubeherd was built for an operator and the people they trust. Renting servers to strangers changes two things: what a customer may do with a server, and how safely a customer gets in. Code that runs in a container is not the problem (every hosting panel works that way); the problem is that today the container is only limited in memory and CPU, a customer with the usual rights can change those limits, and accounts have no second factor and no recovery by e-mail.

> **Core idea**
> A
> **hosting profile**
> for a node that hardens every container, per-server limits that only an operator can change, a split of today's "settings" right so a customer cannot reach the limits, a ready-made
> **Customer**
> role, and the account features hosting needs: two-factor sign-in, password reset by e-mail, optional self-registration with approval, and sign-in notices. All of it is optional and off for existing installs.

Decisions

QUESTION	DECISION
E-mail on sign-in and on registration	Both exist, each switchable by an admin: a notice when someone signs in, and a confirmation link when someone registers.
Limits	Per server, not per customer. An operator (or the API later) sets memory, CPU and disk of a server; the customer cannot change them.
Disk	A per-server disk limit next to memory and CPU, enforced by the agent (section 3).
Own JARs	Customers may upload their own mods and plugins. The hardening of section 2 is what makes that acceptable.
Approving a registration	Only activates the account. It gets no servers; an operator (or the API) creates a server for it and gives the customer a role on that server.
Billing, orders, terms	Outside Cubeherd. An API that creates and suspends servers and accounts follows later.

What stays the same

- Existing installs keep working untouched: the hosting profile is off, accounts mode stays optional, no new right is missing from an existing role (section 4).
- Servers remain plain folders with a compose file; the profile only adds keys to the generated file.

2. Hardening the containers

A node setting **Container profile** (Standard = as today, Hosting) is applied whenever the compose file is regenerated, which already happens on every start. Nothing changes for running servers until their next start.

SETTING IN THE HOSTING PROFILE	WHY
`cap_drop: [ALL]` plus the few capabilities the image needs back	A process in the container loses Linux privileges it does not need. The exact list is found by testing the real image.
`no-new-privileges`	Programs inside cannot gain rights through setuid files.
`pids_limit` (default 2048)	A fork bomb stops at the limit instead of taking the host down. Mods need many threads, so the default is generous and adjustable.
`ulimits` (open files, core size 0)	Bounds descriptor use and prevents core dumps filling the disk.
Log rotation (`json-file` , 10 MB x 3)	A chatty server cannot fill the disk through Docker's log.
`tmpfs` for `/tmp` with a size	Temporary files count against memory, not the data disk.
`init: true`	Zombie processes are reaped.
Optional `runtime` (for example gVisor or Sysbox)	A node operator who installs a stronger runtime selects it here; Cubeherd only writes the key. This is the biggest step against a shared kernel.

- **The compose file stays generated from structured fields.** Customers can only add environment variables (checked keys, quoted values), never compose keys, mounts, ports or privileges. A test tries injection through every free-text field.
- **Network.** Each server is a separate compose project and so has its own network by default (to be verified in the lab). What a container may reach on the host or the LAN is a firewall matter and goes into the hosting guide, together with limiting outgoing traffic against abuse such as attacks launched from a rented server.
- **Not included:** read-only root filesystem (too many mods write outside `/data`), per-customer virtual machines, CAPTCHA. The guide recommends a dedicated VM or node per group of customers who should be separated.

> **Honest limit**
> Containers share the host kernel. The profile makes an attack harder and limits damage from careless or greedy customers; it does not make hostile tenants safe. Before offering paid hosting, the setup should be reviewed by someone who does security reviews, and the website must not call it safe for strangers before that.

3. Limits per server and the disk quota

New and changed fields of a server

FIELD	BEHAVIOUR
Memory, CPU	As today (container limits). Only editable with the new right `server.resources`.
Disk (GB)	New, 0 = unlimited. Set at creation or by an operator, shown to the customer as a bar next to memory.
Port and bind address	Only editable with `server.network`, so a customer cannot move a server onto someone else's port.
Extra environment, Java flags, image tag	Only editable with `server.advanced`.

How the disk limit is enforced

The data folder is a bind mount, so Docker cannot cap it by itself. The agent therefore enforces the limit in steps (the thresholds are node settings):

- Every 30 seconds it measures the usage of servers that have a limit (the measurement exists for the overview and is cached).
- **90 %:** the customer sees a warning.
- **100 %:** Cubeherd refuses what it controls itself: file uploads and saves, world import, add-on downloads, new backups. The server keeps running.
- **120 % for two checks in a row:** the server is stopped and cannot be started until it is below the limit; the reason is shown. A node may choose "block only, never stop".
- For a **hard** limit the hosting guide describes filesystem project quotas (for example XFS) on the host; if they are set up, Cubeherd shows the limit and does not need the watchdog to be the only barrier.

Between two checks a server can write more than its limit. The 120 % rule and the check interval bound that; they are not a promise of an exact byte count.

Suspending

An operator can **suspend** a server: it is stopped, cannot be started, and every user sees why. Suspending an account ends its sessions and blocks sign-in. Both are audited and are what the later API will call for non-payment.

4. Rights and the Customer role

Today `server.settings` covers game rules and also memory, CPU, port and environment, which a customer must not touch. It is split:

RIGHT	ALLOWS	LOWEST SCOPE
`server.settings`	Game rules, MOTD, whitelist switch, view distance, mode and so on	server
`server.resources`	Memory, CPU and disk limit	server
`server.network`	Port and bind address	server
`server.advanced`	Extra environment variables, Java flags, image tag	server
`server.suspend`	Suspend and resume a server	server
`users.approve`	Approve or reject registrations	global

- **Nothing silently changes.** The upgrade gives every role that has `server.settings` the three new rights as well; built-in Owner, Admin and Operator get them too. Only the new Customer role leaves them out.
- **Customer** (built in): view, power, console, commands, players, settings, update, files (read and write), world, add-ons, backups (view, create, restore), statistics. No resources, network, advanced, create, delete or node rights. Customers get it on their server only, never on a node.
- Customers do not create servers themselves. An operator (or the API) creates the server with its limits and grants the role; the customer sees exactly that server and nothing of the others or of the nodes.

5. Safer accounts

Two-factor sign-in (TOTP)

- Standard six-digit time codes that work with any authenticator app. The secret is stored encrypted with the existing key file; each code can be used once; ten single-use recovery codes are shown at setup (stored as hashes).
- Setup page with a QR code (drawn by Cubeherd itself, no service involved) and the key to type in.
- **Policy** (admin setting): optional for everyone, required for roles with sensitive rights (Owner, Admin, anyone who may manage users or nodes), or required for all, with a grace period of a few days to set it up.
- Single sign-on users are not asked twice when the provider already did multi-factor (a setting).
- An admin can reset a user's second factor; that is audited and sends the user a notice.

E-mail: SMTP, verification, reset and notices

- **SMTP settings** (host, port, STARTTLS or TLS, user, password encrypted, sender) with a "send test mail" button. Without them every e-mail feature is off and hidden.
- **Password reset:** a link with a random token, stored as a hash, valid for one hour, usable once. The page never says whether an address is known, requests are rate-limited per address and per IP, a reset ends all sessions, and it is written to the audit log.
- **Verification on registration:** a link that proves the address; an unverified account cannot sign in. Changing an e-mail address starts a new verification. Accounts created by an admin can be marked verified.
- **Sign-in notices** (admin setting: off, new device only, every sign-in): time, device and address and a link "this was not me", which signs the account out everywhere and forces a password reset. A device is recognised by a random cookie, not by IP address.
- Texts follow the interface language of the dashboard; mail is sent directly with a short timeout and failures are logged, with no retry queue in this version.

Registration

MODE (ADMIN SETTING)	EFFECT
Closed (default)	No registration page. Accounts are created by an admin or through sign-in providers.
Open	A registration form; with e-mail configured the address is verified first.
Open with approval	As above, then the account waits in an "Awaiting approval" list until a user with `users.approve` accepts or rejects it. Approving only activates the account (decision: no server, no quota).

- Optional allowed e-mail domains; limits per IP; a new account has no rights; pending and unverified accounts are marked and cannot sign in.
- Not included: CAPTCHA. Approval and the rate limits are the defence against registration spam; a hook for a CAPTCHA provider can follow.

6. Running it behind a reverse proxy

Hosting means a public address. Three things in the current code do not suit a proxy and are fixed first:

- **Secure cookie.** The session cookie gets the `Secure` flag whenever the request came in over HTTPS (directly or through a trusted proxy).
- **Trusted proxies.** A setting lists the proxy addresses whose `X-Forwarded-For` and `X-Forwarded-Proto` are believed. The real client address is then used for sign-in limits, the audit log and sign-in notices; headers from anyone else are ignored.
- **Documentation** with working configurations for nginx, Caddy and Traefik: keep the Host header, switch off buffering for the live console, set HSTS, and keep the agent port (8421) off the proxy and off the internet.

Recommended layout

The dashboard on its own small VM **without** the Docker socket (`CUBEHERD_LOCAL=false`), the game servers on agents, an agent per group of customers who should be isolated. A hacked dashboard then no longer holds Docker on the game host directly.

7. Risks and verification

RISK	MITIGATION
Dropping capabilities breaks the image or a mod	The profile is opt-in per node; tested with Vanilla, Paper, Fabric, Forge, NeoForge and a CurseForge modpack in the lab before it ships
Customer raises their own limits	Split rights, server-side checks on every changed field, and a test that posts each field as a customer
Compose injection through free text	Structured generation, checked keys, tests with hostile values in every field
Disk filled between two checks	Interval and 120 % rule; hard quotas on the host documented
Account takeover	Two-factor, single-use reset tokens, sign-in notices, session revocation, per-user and per-IP limits
Registration spam and mail abuse	Closed by default, approval, verification, rate limits per IP and per address, no reflection of unknown addresses
Mail header injection, token leaks in logs	Addresses and names checked before use; tokens never logged; links carry the token only in the path
A false sense of safety	The hosting guide and the website state the limits (shared kernel, hostile tenants) and recommend a review

Verification plan

- Unit and handler tests for TOTP (known vectors, window, replay), tokens (expiry, reuse, enumeration, timing), registration modes and approval, quotas, the rights split and the upgrade of existing roles, compose generation with the profile (golden files) and injection.
- A small fake SMTP server in the tests; a real mail catcher in the lab.
- **Hostile-tenant lab:** a container that tries to fork without end, write beyond the disk limit, reach other containers and the host, read paths outside `/data`, change its own limits through the dashboard, and gain privileges. Each attempt must fail or be bounded; the result is written into the hosting guide.
- Reverse proxy run with a real proxy: cookies, forwarded addresses, the live console and the limits.

8. Phases

Phase 1	**Proxy and mail foundations** Secure cookie, trusted proxies, SMTP settings and test mail, verified e-mail on accounts, proxy documentation.
Phase 2	**Account security** Password reset, two-factor sign-in with recovery codes and policy, sign-in notices, registration modes with the approval list.
Phase 3	**Rights and limits** The rights split with the upgrade of existing roles, the Customer role, the disk limit with its watchdog, suspending servers and accounts.
Phase 4	**Container profile** The hosting profile, optional runtime, the hostile-tenant lab, the hosting guide with firewall and quota advice.
Later	**API and more** Scoped API tokens for ordering and billing systems, CAPTCHA hook, WebAuthn keys, per-customer overviews.

Order
Phases 1 to 3 are worth having for any installation with outside users. Phase 4 is what makes renting to strangers defensible and needs the most testing, so it comes last but must be done before any paid use.